MAKERERE UNIVERSITY | COLLEGE OF COMPUTING & INFORMATION SCIENCES

MASTER PROJECT BLUEPRINT & TECHNICAL SPECIFICATION

Complete Unified Project Handbook - Embedded Systems Coursework 2026/2027

Kayeera Nathan (24/U/25913/EVE) - Group Leader & System Architect

FULL DISTRIBUTED SYSTEM ARCHITECTURE SPECIFICATION

FULL DISTRIBUTED SYSTEM ARCHITECTURE

Document Owner: Kayeera Nathan (Reg. No: 24/U/25913/EVE)

Role: Group Leader, System Architect, and Integrator

Group: SE-24 | Makerere University (CoCIS) - BSE III & CS II

Course: Embedded Systems Coursework (Academic Year 2026/2027)

Project: Smart Control and Monitoring System for CoCIS

Communication Bus: Wireless ESP-NOW (2.4 GHz Peer-to-Peer Wi-Fi)

Node Designations: ENTERANCE_MASTER, ROOM_NODE_1, ROOM_NODE_2

Associated Diagram: SYSTEM_ARCHITECTURE_DIAGRAM.svg

1. Architectural Philosophy & Overview

The CoCIS Smart Control and Monitoring System is engineered as a distributed, multi-tier embedded IoT architecture. Rather than running single-point monolithic wiring across campus facilities, the design partitions responsibilities across three autonomous ESP32 microcontrollers communicating wirelessly via 2.4 GHz ESP-NOW:

- 1. Central Gateway & Access Terminal (ENTERANCE_MASTER - Master ESP32):
- Controls physical building ingress and egress with dual RFID readers and servo barriers.
- Enforces building security policies: configurable schedule curfew, anti-passback, and intrusion lockout.
- Serves as the single authority for non-volatile SD storage (FATFS), preventing file access conflicts.
- Hosts the administrator Serial CLI management console over USB-UART0.
- Operates a FreeRTOS real-time synchronization engine with priority queues and mutexes.
- 2. Autonomous Lecture Room Nodes (ROOM_NODE_1 & ROOM_NODE_2 - Edge ESP32s):
- Continuously evaluate local occupancy using PIR motion sensors and ambient light sensors (LDR).
- Automatically control lecture room lighting based on motion and daylight availability.
- Stream telemetry (occupancy, light relay state, lux level, energy metrics) wirelessly to ENTERANCE_MASTER every 2 seconds.
- Respond immediately to Master broadcast commands (such as campus-wide emergency evacuations).

2. Full System Architecture Diagram (Mermaid)

```
flowchart TD
    subgraph EntranceMaster["ENTERANCE_MASTER: Entrance Master & Gateway (ESP32 #1)"]
        direction TB
```

```
        subgraph PhysicalEntry["Physical Entry Section"]
            RFID_IN["RFID Entry Reader\n(RC522 - VSPI)"]
            SERVO_IN["Entry Door Servo\n(GPIO 13)"]
            LED_GREEN["Access Granted LED\n(GPIO 2)"]
        end

        subgraph PhysicalExit["Physical Exit Section"]
            RFID_OUT["RFID Exit Reader\n(RC522 - Shared VSPI)"]
            SERVO_OUT["Exit Door Servo\n(GPIO 15)"]
            LED_RED["Access Denied LED\n(GPIO 4)"]
            BUZZER["Curfew/Alarm Buzzer\n(GPIO 12)"]
        end

        subgraph FreeRTOSCore["ESP32 Dual-Core RTOS & Concurrency Engine"]
            direction TB
            subgraph Tasks["FreeRTOS Tasks"]
                T_ACCESS["AccessControlTask (Core 1)"]
                T_LOG["SDLoggerTask (Core 0)"]
                T_WIFI["WirelessRxTask (Core 1)"]
                T_CLI["SerialCLITask (Core 0)"]
                T_HB["HeartbeatMonitor (Core 1)"]
            end
            subgraph Queues["IPC Queues"]
                Q_ACCESS["xAccessEventQueue"]
                Q_LOG["xSDLogQueue"]
                Q_WIFI["xWirelessPacketQueue"]
            end
            subgraph Mutexes["Thread-Safe Mutexes"]
                M_OCC["xOccupancyMutex"]
                M_SCHED["xScheduleMutex"]
                M_SD["xSDCardMutex"]
                M_ROOM["xRoomStateMutex"]
            end
        end

        subgraph StorageSubsystem["Storage Subsystem (MicroSD SPI)"]
            F_USERS["users.csv\n(Tags & Roles)"]
            F_LOGS["access_log.csv\n(Audit Trail)"]
            F_OCC["occupancy.csv\n(Crash Recovery)"]
            F_CONF["config.txt\n(Schedule & Lockout)"]
        end

        subgraph ManagementConsole["Serial CLI Interface"]
            CLI["USB-UART0 Console\n(115200 Baud)"]
            AUTH["SHA-256 Login &\nSession Manager"]
        end
    end

    subgraph WirelessBus["2.4 GHz ESP-NOW Wireless Protocol Bus (Connectionless Peer-to-Peer)"]
        direction TB
        WIFI_TRANS["RF Transceiver (WIFI_MODE_STA)\nSub-5ms Latency | Hardware MAC ACKs | No Ro
        PKT_TELEMETRY["Telemetry Packets: occupied, lights_on, lux, energy_sec"]
        PKT_EMERGENCY["Broadcast Alert: Instant Evacuation (0xFF:FF:FF:FF:FF:FF)"]
    end

    subgraph RoomNode1["ROOM_NODE_1: Lecture Room 1 (ESP32 #2)"]
        PIR_1["PIR Motion Sensor\n(GPIO 14 - Interrupt)"]
        LDR_1["Ambient Light LDR\n(GPIO 34 - ADC1)"]
        RELAY_1["Lighting Relay / LED\n(GPIO 12)"]
        HB_LED_1["Heartbeat LED\n(GPIO 2)"]
        CTRL_1["Energy Logic: 30s Countdown\n+ Daylight Thresholding"]
    end

    subgraph RoomNode2["ROOM_NODE_2: Lecture Room 2 (ESP32 #3)"]
        PIR_2["PIR Motion Sensor\n(GPIO 14 - Interrupt)"]
```

```
        LDR_2["Ambient Light LDR\n(GPIO 34 - ADC1)"]
        RELAY_2["Lighting Relay / LED\n(GPIO 12)"]
        HB_LED_2["Heartbeat LED\n(GPIO 2)"]
        CTRL_2["Energy Logic: 30s Countdown\n+ Daylight Thresholding"]
    end

    %% Physical to Tasks
    RFID_IN -->|Swipe Event| Q_ACCESS
    RFID_OUT -->|Swipe Event| Q_ACCESS
    Q_ACCESS --> T_ACCESS
    T_ACCESS -->|Grant/Deny| SERVO_IN
    T_ACCESS -->|Grant/Deny| SERVO_OUT
    T_ACCESS -->|Feedback| LED_GREEN
    T_ACCESS -->|Feedback| LED_RED
    T_ACCESS -->|Lockout/Curfew| BUZZER

    %% Concurrency Interactions
    T_ACCESS <-->|Lock/Verify| M_OCC
    T_ACCESS <-->|Lock/Verify| M_SCHED
    T_ACCESS -->|Push Event| Q_LOG
    Q_LOG --> T_LOG
    T_LOG <-->|Lock/Write| M_SD
    M_SD --> StorageSubsystem

    %% CLI Interactions
    CLI --> AUTH
    AUTH --> T_CLI
    T_CLI <-->|Read Occupants| M_OCC
    T_CLI <-->|Update Window| M_SCHED
    T_CLI <-->|Query Rooms| M_ROOM

    %% Wireless Links
    T_WIFI <--> WIFI_TRANS
    T_HB -->|Ping/Check| WIFI_TRANS
    WIFI_TRANS <--> RoomNode1
    WIFI_TRANS <--> RoomNode2
    RoomNode1 -->|2s Telemetry| WIFI_TRANS
    RoomNode2 -->|2s Telemetry| WIFI_TRANS
    WIFI_TRANS -->|Evacuate Broadcast| RoomNode1
    WIFI_TRANS -->|Evacuate Broadcast| RoomNode2
```

3. Node-by-Node Technical Architecture

3.1 ‘ENTERANCE_MASTER‘ - Entrance Terminal & Gateway

- Physical Ingress/Egress Isolation: The entrance is split into dedicated Entry and Exit corridors. Each corridor has its own RFID reader (RC522 over SPI with dedicated Chip Select pins: GPIO 21 for Entry, GPIO 5 for Exit) and independent servo barrier gates.
- Strict Schedule Curfew (Coursework Section 2.6): When current time is outside authorized hours (e.g., 20:01 to 06:59), the state machine rejects both entry and exit attempts unless the tag possesses admin or security role credentials. Unauthorized exit attempts trigger the buzzer (GPIO 12) and write an AFTER_HOURS_VIOLATION record to access_log.csv.
- Anti-Passback Enforcement: Prevents card sharing. A tag recorded as "Inside" cannot generate an Entry event; a tag recorded as "Outside" cannot generate an Exit event.
- Intrusion Lockout: Three consecutive unregistered or rejected card swipes trigger a 60-second hardware lockout, pulsing the alarm buzzer and preventing brute-force access.
- Single-Writer SD Card Subsystem: Only ENTERANCE_MASTER writes to the MicroSD card. This completely eliminates multi-board race conditions and file lock contentions.

3.2 FreeRTOS Concurrency & Multithreading Layer

To fulfill the requirements of Section 2.7 (Resource Synchronization and System Reliability), ENTERANCE_MASTER employs a multi-threaded FreeRTOS architecture:

```
[Hardware ISRs / Inbound Events]
       RFID Swipe ISR --------> xAccessEventQueue (Depth 10) ------> AccessControlTask (Core 1)
       ESP-NOW Wireless ISR --> xWirelessPacketQueue (Depth 15) ---> WirelessRxTask (Core 1)
       Serial Rx ISR ---------> CLI Parser Buffer -----------------> SerialCLITask (Core 0)
                                                                            |
[Shared State Protection]                                                   |
       xOccupancyMutex -------- Guards in-memory occupancy array <----------+
       xScheduleMutex --------- Guards 07:00-20:00 operating window <-------+
       xRoomStateMutex -------- Guards Room 1 & 2 cached metrics <----------+
       xSDCardMutex ----------- Guards FATFS physical SPI bus <-------------+
                                                                            |
[Non-Blocking File Persistence]                                            |
       AccessControlTask -----> xSDLogQueue (Depth 20) ------------> SDLoggerTask (Core 0)
                                                                            |
                                                                  MicroSD Card (SPI)
```

3.3 ‘ROOM_NODE_1‘ & ‘ROOM_NODE_2‘ - Distributed Lecture Room Controllers

- PIR Motion Sensing & Debounce: Digital input on GPIO 14 triggers an interrupt on motion. A software timer resets a 30-second no-motion countdown. Only when countdown reaches zero does the light relay open.
- Daylight-Aware Illumination (Innovation): An analog LDR photocell on ADC1_CH6 (GPIO 34) measures ambient light. Even if the room is occupied, if ambient light exceeds the threshold (e.g. >250 lux from sunlight), lights remain OFF, saving substantial electricity.
- Autonomous Local Control: Room nodes operate independently. If ENTERANCE_MASTER goes offline, lecture rooms continue sensing motion and switching lights without interruption.
- Wireless Telemetry & Keep-Alive: Every 2 seconds, each room node bundles its status into an ESP-NOW packet:

```
typedef struct __attribute__((packed)) {
    uint8_t  occupied;        // 1 = Occupied, 0 = Vacant
    uint8_t  lights_on;       // 1 = Relay Closed, 0 = Relay Open
    uint16_t light_level_lux; // ADC converted ambient illumination
    uint32_t seconds_active;  // Energy saving counter
} room_status_payload_t;
```

4. Wireless Communication Protocol (ESP-NOW)

Parameter	Specification	Technical Justification
RF Frequency	2.4 GHz ISM Band	Native ESP32 integrated radio; zero external hardware
Protocol Type	ESP-NOW (802.11 Vendor Action Frames)	Connectionless, peer-to-peer; no router, DHCP, or SSID required
Latency	< 4 milliseconds	Immediate response for telemetry streaming and emergency overrides
Addressing	6-Byte Hardware MAC Addressing	Unicast peer registration (ENTERANCE_MASTER, ROOM_NODE_1, ROOM_NODE_2) + Broadcast
Delivery Guarantee	Hardware MAC ACKs + 3x Software Retry	Automatic status reported via esp_now_register_send_cb
Broadcast Channel	FF:FF:FF:FF:FF:FF	Used for zero-latency campus emergency evacuation commands

5. End-to-End Operational Pipelines

Pipeline 1: Student / Staff RFID Ingress

- 1. Tag tapped at Entry Reader $\rightarrow$ SPI interrupt reads UID.
- 2. xAccessEventQueue delivers UID to AccessControlTask.
- 3. Task acquires xScheduleMutex: Verifies current time $\in [07:00, 20:00]$.
- 4. Task verifies users.csv: Tag exists and is authorized.
- 5. Task acquires xOccupancyMutex: Anti-passback check confirms user is currently "Outside".
- 6. User marked "Inside" in memory; xSDLogQueue receives log record (ENTRY, GRANTED).
- 7. GPIO 13 pulses PWM signal $\rightarrow$ Entry servo opens 90 for 5 seconds $\rightarrow$ Green LED blinks.
- 8. SDLoggerTask wakes up, acquires xSDCardMutex, and appends event to access_log.csv.

Pipeline 2: Strict Curfew Lockdown (After-Hours Exit Attempt)

- 1. Tag tapped at Exit Reader at 21:15 $\rightarrow$ UID passed to AccessControlTask.
- 2. Task acquires xScheduleMutex: Current time (21:15) exceeds schedule_end (20:00).
- 3. Task checks user role in users.csv: Role is student (Not Admin).
- 4. Access Denied: Exit servo remains rigidly locked.
- 5. Red LED illuminates; GPIO 12 activates buzzer warning.
- 6. Audit log written to access_log.csv: CURFEW_LOCKDOWN_VIOLATION.

Pipeline 3: Wireless Emergency Evacuation

- 1. Physical button pressed (GPIO 32) OR admin enters emergency on in Serial CLI.
- 2. ENTERANCE_MASTER immediately forces Entry and Exit servos to 90 (unlocked).
- 3. ENTERANCE_MASTER broadcasts MSG_TYPE_EMERGENCY_ALERT over ESP-NOW to FF:FF:FF:FF:FF:FF.
- 4. ROOM_NODE_1 & ROOM_NODE_2 receive broadcast: Override light relays to FORCED ON (illuminating corridors and lecture halls for safe evacuation).
- 5. ENTERANCE_MASTER flushes occupancy.csv and logs CAMPUS_EMERGENCY_EVACUATION.

STEP-BY-STEP ACTION PLAN & TECHNICAL ROADMAP

STEP-BY-STEP ACTION PLAN & TECHNICAL ROADMAP

Student: Kayeera Nathan (Reg. No: 24/U/25913/EVE)

Role: Group Leader, System Architect, and Integrator

Group: SE-24 | Makerere University (CoCIS) - BSE III & CS II

Course: Embedded Systems Coursework (Academic Year 2026/2027)

Project: Smart Control and Monitoring System for CoCIS

Communication Bus: Wireless ESP-NOW (2.4 GHz Peer-to-Peer Wi-Fi)

Submission Deadline: Monday, 2 November 2026 (Submit on MUELE)

Presentation / Viva: Monday, 2 November 2026 (In-person attendance mandatory)

1. Executive Summary & Your Dual Role

As Group Leader, System Architect, and Integrator, you have two equally vital responsibilities:

- 1. Technical Leadership & Architecture: You own the overarching system design, the wireless ESP-NOW (2.4 GHz Wi-Fi) communication layer, FreeRTOS concurrency and synchronization architecture, emergency evacuation logic, offline heartbeat monitoring, and the master integration of all 5 group members' code on ESP-IDF and Wokwi.
- 2. Project & Submission Governance: You are responsible for ensuring the group meets the 2 November deadline, freezing code on time, compiling the final comprehensive report, verifying all individual video links and Wokwi simulation links, and preparing the team to defend their work before the lecturer.

TIP / BEST PRACTICE

Why ESP-NOW Wi-Fi is our Primary Architectural Choice: In a real-world facility like CoCIS, laying hundreds of meters of physical UART wires between the main entrance and multiple lecture rooms across corridors is prone to electrical noise, signal degradation, and high installation costs. We replace physical serial cables with **ESP-NOW**, Espressif's connectionless 2.4 GHz Wi-Fi protocol. This operates without a router or internet access, delivers sub-5ms latency, includes hardware MAC-layer acknowledgments, and directly satisfies the coursework's heavy emphasis on **system innovation**.

2. Master Step-by-Step Execution Plan (Day-by-Day)

Phase 0: Kickoff, Setup & Interface Contract (Oct 23 - Oct 24) - *CURRENT STAGE*

☐ Step 0.1: Establish Team Governance & Confirm Roles

- Hold a kickoff call/chat with all 4 group members:
- Kasozi David Paul (24/U/25905/EVE): Entrance Terminal (RFID, doors, anti-passback, lockout).
- Zziwa Mark Alex (24/U/11978/EVE): Room Terminals (PIR motion, lighting relays, LDR light sensing, ESP-NOW transmitter).
- Sseku Jovan (24/U/11172/EVE): Storage & Data Management (SD card, CSV logs, timestamps, crash recovery).
- Namuzibwa Laurinda C (24/U/1472): CLI interface, Authentication, Security, QA & Test Plan.
- Set a fixed 15-minute daily standup meeting (e.g., 8:00 PM daily).

☐ Step 0.2: Initialize the Version Control Repository

- Create a private GitHub repository named SE-24-CoCIS-Smart-System.
- Set up directory structure:

```
components/common_protocol/ # Shared headers, ESP-NOW protocol, data types, FreeRTOS sy
src/
    ENTERANCE_MASTER/       # Entrance Master (ESP32 #1: RFID, SD, CLI, ESP-NOW Gateway
    ROOM_NODE_1/            # Lecture Room 1 (ESP32 #2 + ESP-NOW)
    ROOM_NODE_2/            # Lecture Room 2 (ESP32 #3 + ESP-NOW)
docs/                       # Diagrams, design notes, test logs
simulation/                 # Wokwi diagram.json, wokwi.toml, and virtual SD card
```

- Create dedicated feature branches for each member (feat/entrance-david, feat/rooms-mark, feat/storage-jovan, feat/cli-laurinda, feat/arch-nathan). Enforce a rule: *no direct pushes to main*.

☐ Step 0.3: Environment Setup & Wokwi Wireless Feasibility Check

- Confirm every member has ESP-IDF installed and can run a basic C application in Wokwi.
- Critical Technical Feasibility Checks for you to supervise:
- *Multi-ESP32 ESP-NOW Simulation:* Set up two ESP32 boards in Wokwi's diagram.json. Wokwi simulates the 2.4 GHz radio space automatically-no inter-board connecting wires are required! Run a simple ESP-NOW sender/receiver test to verify packet delivery in browser simulation.
- *RFID in Wokwi:* Verify David's RFID approach. If SPI MFRC522 has simulation constraints in Wokwi, implement the approved fallback: UART/Serial RFID tag emulation or keypad input, clearly documented.

- *SD Card in Wokwi:* Verify Jovan's SPI SD card simulation with FATFS/VFS.

☐ Step 0.4: Author and Freeze the Wireless Interface Contract ('design.md')

- Write down the exact pin allocations, ESP-NOW packet framing format, MAC address mapping, CSV data schemas, and FreeRTOS queue definitions.
- Freeze this document by Saturday midnight so your teammates can write independent code without integration headaches.

Phase 1: Nathan's Core Technical Engineering (Oct 25 - Oct 27)

During this phase, each teammate builds their unit modules. Here is your specific coding workload:

☐ Step 1.1: Develop the Wireless ESP-NOW Protocol Layer ('common/esp_now_transport.h' & '.c')

- Implement the connectionless Wi-Fi stack initialization:
- nvs_flash_init() (mandatory for ESP32 Wi-Fi RF calibration).
- esp_netif_init() and default event loop creation.
- Wi-Fi initialization in Station mode (WIFI_MODE_STA) and start RF without connecting to an SSID.
- esp_now_init() and registration of send and receive callbacks.
- Implement Peer Management:
- Register Master MAC address in Room nodes.
- Register Room 1 and Room 2 MAC addresses in Master node.
- Implement custom message frames within ESP-NOW payload (up to 250 bytes):
- MSG_ROOM_STATUS (Room -> Master: Occupied flag, Light flag, Lux level, energy timer).
- MSG_LIGHT_OVERRIDE (Master -> Room: Auto, Force OFF, Force ON).
- MSG_HEARTBEAT (Room -> Master: Periodic keep-alive ping every 2 seconds).
- MSG_EMERGENCY_ALARM (Master -> Broadcast: Instant evacuation alert).
- Implement Delivery Verification:
- Utilize ESP-NOW's built-in send_cb status (ESP_NOW_SEND_SUCCESS vs ESP_NOW_SEND_FAIL).
- Add software sequence IDs and automatic 3x retry on packet failure.

☐ Step 1.2: Build the Thread-Safe Concurrency & Synchronization Module ('common/sync_manager.h')

- The assignment explicitly grades Resource Synchronization and System Reliability (Section 2.7). You must build this properly using FreeRTOS primitives:
- Mutexes:
- xOccupancyMutex: Protects the in-memory array of persons currently inside the building.
- xScheduleMutex: Protects current operating hours and configuration state.
- xSDCardMutex: Protects shared SPI bus and file operations.
- xRoomStateMutex: Protects the cached telemetry states of Room 1 and Room 2.
- Queues:
- xAccessEventQueue: David's RFID task pushes entry/exit requests -> Master logic evaluates -> logs to SD.
- xSDLogQueue: Dedicated queue consumed by Jovan's SD logging task to eliminate disk write bottlenecks.
- xWirelessPacketQueue: ESP-NOW receive callback copies incoming packets into this queue, allowing the master processing task to handle wireless telemetry asynchronously without blocking Wi-Fi interrupts.

☐ Step 1.3: Code the Wireless Heartbeat & Offline Detection Engine

- Master runs a periodic FreeRTOS task (heartbeat_monitor_task, runs every 2-3 seconds).
- Tracks timestamp of last received wireless packet from Room 1 and Room 2.
- If no ESP-NOW heartbeat is received within 10 seconds, trigger WIRELESS_NODE_OFFLINE on the master console and flag the room as offline.

☐ Step 1.4: Conduct Peer Reviews (Tuesday Evening, Oct 27)

- Review David's Entrance code, Mark's Room code, Jovan's SD Card code, and Laurinda's CLI/Auth code.
- Verify that no teammate is writing directly to shared memory without using your mutexes, and all inter-task communication uses your FreeRTOS queues.

Phase 2: Master Node Integration & System Synchronization (Oct 28 - Oct 29)

- [ ] Step 2.1: Merge Teammate Modules into Node A (Master)
  - Integrate David's RFID reader & door servo control into the master processing loop.
  - Connect Laurinda's Serial CLI parser to your synchronized data tables.
  - Connect Jovan's SD card manager so all door transitions write safely to access_log.csv.
- [ ] Step 2.2: Implement the Strict Building Access Schedule Logic
  - Coursework Requirement (Section 2.6):
  - Configurable time window (e.g., 07:00 to 20:00).
  - Outside operating hours: Entry is denied.
  - Crucial Rule: People inside the building CANNOT leave outside authorized hours (strict lockdown prevention against unauthorized after-hours asset removal).
  - Exception: Authenticated Admin / Security override.
- [ ] Step 2.3: Integrate Wireless Room Telemetry with CLI Dashboard
  - Connect incoming ESP-NOW packets from Room 1 & Room 2 to your local state cache.
  - Master dashboard commands (rooms, lights, status) display live wireless telemetry received from the lecture room microcontrollers.
- [ ] Step 2.4: Execute Concurrency & Race-Condition Stress Tests
  - Test simultaneous entry and exit scans happening at the exact same millisecond.
  - Test a serial CLI query (occupancy or logs) while an SD card write is actively executing and an ESP-NOW wireless packet arrives.
  - Ensure zero deadlocks, zero buffer corruptions, and zero missed log entries.

Phase 3: Innovation Features, Hardening & Code Freeze (Oct 29 - Oct 30)

The coursework places particular emphasis on innovative features. You lead and verify these:

- [ ] Step 3.1: Finalize Your Personal Innovation - Wireless Emergency Evacuation Mode
  - CLI command emergency activate or a dedicated physical emergency pushbutton.
  - Instantly unlocks both door servos (servos swing to 90 open position).
  - Broadcasts wireless MSG_EMERGENCY_ALARM via ESP-NOW to all lecture rooms (forces all room lights ON for emergency evacuation visibility).
  - Flushes occupancy records to SD card with an EMERGENCY_EVACUATION event log.
  - Command emergency reset broadcasts normal status and restores regular schedule.
- [ ] Step 3.2: Coordinate Teammate Innovations
  - Wireless ESP-NOW Smart Mesh (Nathan): Cable-free building automation network with sub-5ms latency and automatic peer health monitoring.
  - Anti-passback (David & Jovan): Tag cannot enter if already inside; cannot exit if already outside.
  - Intrusion Lockout (David & Laurinda): 3 consecutive invalid RFID scans trigger buzzer alarm and lock terminal for 60 seconds.
  - Daylight-Aware Smart Lighting (Mark): If room has high ambient light (LDR reading high), lights remain off even if PIR senses occupancy.
  - Crash Recovery (Jovan): On master reboot/power-cut, master reads occupancy.csv and config.txt to restore active occupants and schedule without data loss.
  - Energy Savings Analytics (Mark & Jovan): Calculates and prints kilowatt-hours/minutes saved when lights were shut off due to no occupancy.
- [ ] Step 3.3: Complete Full System Verification Against Test Matrix
  - Execute all test cases with Laurinda (Valid entry, Invalid exit, Anti-passback trigger, Outside schedule entry/exit attempt, Wireless packet packet drop test, Power reboot recovery, SD card removal handling).

- [ ] Step 3.4: HARD CODE FREEZE - Friday, 30 October 2026 at 8:00 PM
  - Merge all stable feature branches into main.
  - Lock the repository. *No code edits after this point except urgent bug fixes.*

Phase 4: Deliverables Assembly, Video Recordings & Report (Oct 30 - Nov 1)

According to Section 4 of the Coursework PDF, each group must submit 5 mandatory deliverables. Here is your checklist:

- [ ] Deliverable 1: Comprehensive System Operation Report
  - Coordinate report compilation with Laurinda:
  - *Nathan's written sections:* Section 1: System Overview & Architecture; Section 2: Wireless ESP-NOW Protocol & Peer Network; Section 3: FreeRTOS Concurrency & Synchronization; Section 6: Wireless Emergency Evacuation & Heartbeat Innovations; Section 7: System Integration & Master Wokwi Setup.
  - Include block diagrams, state machine flowcharts, circuit schematics, and concurrency mutex diagrams.
  - Include the table of group members and clear individual contributions.
- [ ] Deliverable 2: Public Wokwi Simulation Link
  - Build the clean, consolidated Wokwi project with all 3 ESP32 nodes running wirelessly.
  - Ensure the simulation link is set to Public and test it in an Incognito/Private browser window.
- [ ] Deliverable 3: Individual Video Demonstrations (Screen Recordings)
  - Each member MUST record their own 3 to 5-minute video.
  - Nathan's Personal Video Script (3-5 min):
  - 1. *Introduction (30s):* Name, Reg No, Role (Leader & Architect), brief overview of the 3-node distributed wireless system.
  - 2. *Architecture & Synchronization (1.5 min):* Show the block diagram, explain the FreeRTOS mutexes, message queues, and why synchronization prevents race conditions.
  - 3. *Wireless ESP-NOW Protocol & Heartbeat (1 min):* Show the C code in esp_now_transport.c, demonstrate packet send/receive callbacks, hardware MAC ACKs, and wireless node offline detection.
  - 4. *Live Wokwi Demonstration (1.5 min):* Trigger the Emergency Evacuation Mode live in Wokwi, show doors unlocking, wireless broadcast packet triggering lecture room lights ON, and emergency log entry written to SD card.
  - Upload your video to Google Drive / YouTube (unlisted) and ensure link access is set to *"Anyone with the link can view"*.
- [ ] Deliverable 4: Individual Contributions Section
  - Ensure your written report explicitly details every member's exact contribution to avoid mark deductions.
- [ ] Deliverable 5: Group Details Table
  - Ensure all 5 names, registration numbers, and roles are properly formatted on the title page.
- [ ] Step 4.5: Conduct Team Mock Defense (Sunday, 1 Nov)
  - Run a mock Q&A session. Quiz David on anti-passback, Mark on PIR debounce and ESP-NOW transmissions, Jovan on FATFS queues, and Laurinda on SHA/hash authentication.
  - Ensure everyone understands how the wireless system fits together.

Phase 5: Final Submission & Lecturer Defense (Nov 1 - Nov 2)

- [ ] Step 5.1: Final MUELE Submission (Sunday Night / Early Monday Morning)
  - Export the report as a clean PDF (SE24_CoCIS_Smart_Control_System_Report.pdf).
  - Verify all 5 video links and the Wokwi link are live and clickable inside the PDF.
  - Upload the final report to MUELE well before the deadline on 2nd November 2026.
- [ ] Step 5.2: In-Person Presentation & Defense (Monday, 2 November 2026)
  - Ensure 100% group attendance (as stated in coursework instructions, absent members get zero marks).
  - Open the live Wokwi simulation on a laptop, ready to demonstrate.
  - Confidently explain your architecture, wireless ESP-NOW peer-to-peer design, synchronization mechanisms, and leadership coordination.

3. Kayeera Nathan’s Technical Specification & Code Blueprint

3.1 Wireless ESP-NOW Protocol Definition (‘common/protocol.h‘)

```
#ifndef PROTOCOL_H
#define PROTOCOL_H

#include <stdint.h>
#include <stdbool.h>

#define ESP_NOW_PAYLOAD_MAGIC 0xAA55

typedef enum {
    NODE_MASTER  = 0x01,
    NODE_ROOM_1  = 0x02,
    NODE_ROOM_2  = 0x03
} node_id_t;

typedef enum {
    MSG_TYPE_HEARTBEAT       = 0x01,
    MSG_TYPE_ROOM_STATUS     = 0x02,
    MSG_TYPE_LIGHT_OVERRIDE  = 0x03,
    MSG_TYPE_EMERGENCY_ALERT = 0x04
} msg_type_t;

// ESP-NOW Wireless Packet Frame (Fits easily within ESP-NOW 250-byte max)
typedef struct __attribute__((packed)) {
    uint16_t magic;           // 0xAA55 Protocol Identifier
    uint8_t  msg_id;          // Sequence ID for tracking
    uint8_t  sender_id;       // Node ID of sender
    uint8_t  target_id;       // Node ID of receiver (or 0xFF for broadcast)
    uint8_t  msg_type;        // Type of message
    uint8_t  payload_len;     // Length of payload
    uint8_t  payload[32];     // Application payload
    uint8_t  checksum;        // XOR checksum for payload integrity
} esp_now_packet_t;

// Room status payload structure
typedef struct __attribute__((packed)) {
    uint8_t  occupied;        // 1 = Occupied, 0 = Empty
    uint8_t  lights_on;       // 1 = ON, 0 = OFF
    uint16_t light_level_lux; // Ambient brightness from LDR
    uint32_t seconds_active;  // Energy saving counter
} room_status_payload_t;

#endif
```

3.2 FreeRTOS Concurrency Architecture (‘src/ENTERANCE_MASTER/main.c‘)

```
/*
 * CONCURRENCY CONTROL & WIRELESS PACKET DISPATCH
 * Designed by: Kayeera Nathan (System Architect, SE-24)
 */

#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/semphr.h"
#include "freertos/queue.h"
#include "esp_now.h"
#include "protocol.h"
```

```
// Mutexes for Thread Safety
SemaphoreHandle_t xOccupancyMutex;
SemaphoreHandle_t xScheduleMutex;
SemaphoreHandle_t xSDCardMutex;
SemaphoreHandle_t xRoomStateMutex;

// Event and Log Queues
QueueHandle_t xAccessEventQueue;
QueueHandle_t xSDLogQueue;
QueueHandle_t xWirelessPacketQueue;

typedef struct {
    char tag_id[16];
    char user_name[32];
    uint8_t direction;        // 0 = ENTRY, 1 = EXIT
    uint32_t timestamp;
} access_event_t;

void init_concurrency_objects(void) {
    xOccupancyMutex      = xSemaphoreCreateMutex();
    xScheduleMutex       = xSemaphoreCreateMutex();
    xSDCardMutex         = xSemaphoreCreateMutex();
    xRoomStateMutex      = xSemaphoreCreateMutex();

    xAccessEventQueue    = xQueueCreate(10, sizeof(access_event_t));
    xSDLogQueue          = xQueueCreate(20, sizeof(access_event_t));
    xWirelessPacketQueue = xQueueCreate(15, sizeof(esp_now_packet_t));
}

// Wireless ESP-NOW RX Callback - pushes packet to queue without blocking Wi-Fi
void on_esp_now_data_recv(const esp_now_recv_info_t *info, const uint8_t *data, int len) {
    if (len == sizeof(esp_now_packet_t)) {
        esp_now_packet_t pkt;
        memcpy(&pkt, data, sizeof(esp_now_packet_t));
        BaseType_t xHigherPriorityTaskWoken = pdFALSE;
        xQueueSendFromISR(xWirelessPacketQueue, &pkt, &xHigherPriorityTaskWoken);
        if (xHigherPriorityTaskWoken) {
            portYIELD_FROM_ISR();
        }
    }
}
```

4. Immediate Next Steps (To Do Today, Oct 24)

- 1. Share the role confirmation with the group chat.
- 2. Initialize the GitHub repository SE-24-CoCIS-Smart-System.
- 3. Save the frozen INTERFACE_CONTRACT_AND_DESIGN.md specification and announce the upgrade to ESP-NOW Wi-Fi.
- 4. Test the multi-ESP32 ESP-NOW simulation in Wokwi (no interconnecting wires needed!).

SYSTEM ARCHITECTURE & INTERFACE CONTRACT (design.md)

SYSTEM ARCHITECTURE & INTERFACE CONTRACT (design.md)

Document Owner: Kayeera Nathan (System Architect, SE-24)

Project: Smart Control and Monitoring System for CoCIS

Target Platform: ESP32 (ESP-IDF) + Wokwi Simulation

Communication Bus: Wireless ESP-NOW (2.4 GHz Peer-to-Peer Wi-Fi)

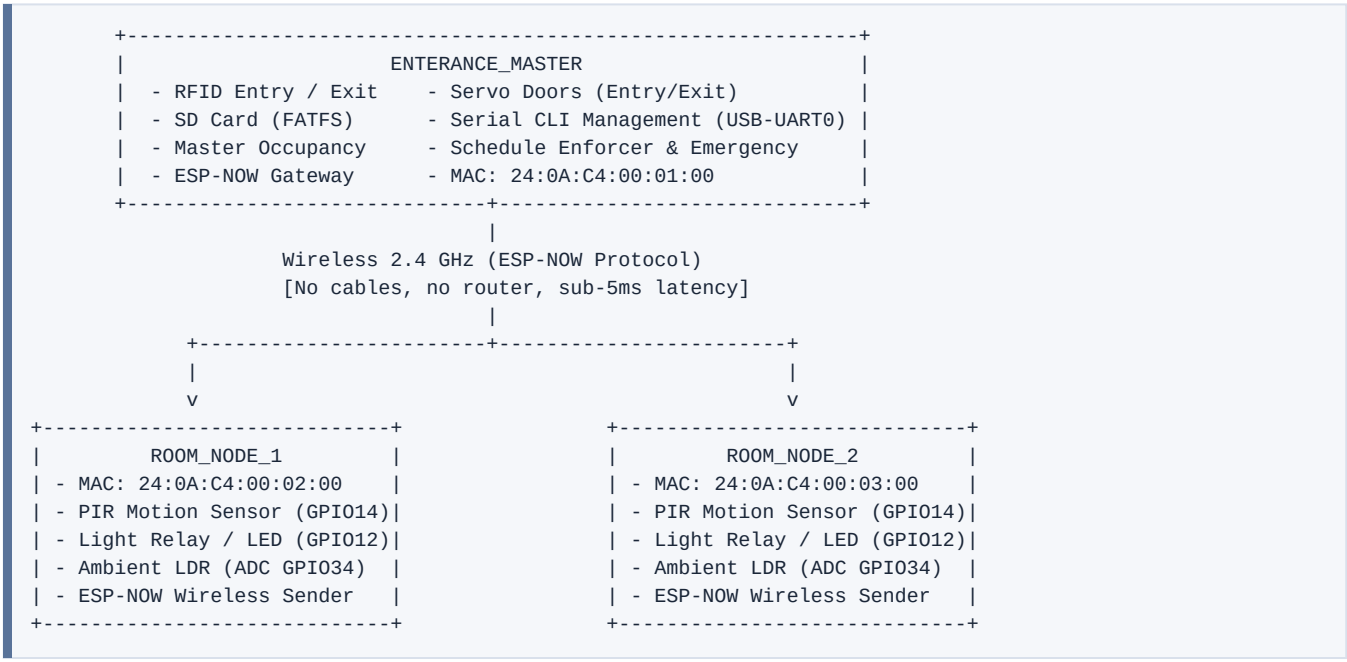
Node Designations: ENTERANCE_MASTER, ROOM_NODE_1, ROOM_NODE_2

Status: Frozen for Team Implementation

1. System Topology Overview (Wireless Star Architecture)

The system consists of three ESP32 microcontroller nodes communicating wirelessly using ESP-NOW (2.4 GHz connectionless Wi-Fi):

- 1. ENTERANCE_MASTER (Entrance Terminal & Central Gateway - ESP32 #1):
- Handles entry & exit RFID scanners, servo door actuators, buzzer, and feedback LEDs.
- Hosts the SD Card module (SPI/FATFS) for logging, user database, and configuration storage.
- Provides the Serial CLI over USB-UART0 for administrative and monitoring access.
- Acts as the central synchronization master and wireless gateway, receiving telemetry from Room nodes and dispatching lighting/emergency overrides via ESP-NOW.
- 2. ROOM_NODE_1 (Lecture Room 1 Controller - ESP32 #2):
- PIR motion detection, ambient light sensing (LDR), and light relay control.
- Transmits room occupancy, lux levels, and lighting status to ENTERANCE_MASTER wirelessly via ESP-NOW.
- 3. ROOM_NODE_2 (Lecture Room 2 Controller - ESP32 #3):
- Identical hardware setup to ROOM_NODE_1; manages Lecture Room 2 and transmits wireless telemetry to ENTERANCE_MASTER.



2. Hardware Pin Allocations

ENTERANCE_MASTER Pin Mapping (ESP32 #1)

Component / Function	ESP32 GPIO Pin	Description / Interface
RFID Entry Reader	GPIO 21 (SDA), GPIO 18 (SCK), GPIO 23 (MOSI), GPIO 19 (MISO), GPIO 22 (RST)	SPI Bus / RC522 (or Serial Emulation)
RFID Exit Reader	GPIO 5 (SDA), Shared SPI Bus (SCK:18, MOSI:23, MISO:19), GPIO 4 (RST)	SPI Secondary Chip Select

Entry Door Actuator	GPIO 13	PWM / Servo Motor (0 Closed, 90 Open)
Exit Door Actuator	GPIO 15	PWM / Servo Motor (0 Closed, 90 Open)
Access Granted LED	GPIO 2	Green LED (Active High)
Access Denied LED	GPIO 4	Red LED (Active High)
Alarm Buzzer	GPIO 12	Active Buzzer (Lockout & Alarm)
SD Card CS	GPIO 27	SPI CS (Shared VSPI or Dedicated HSPI)
SD Card MOSI	GPIO 23	SPI MOSI
SD Card MISO	GPIO 19	SPI MISO
SD Card CLK	GPIO 18	SPI SCK
Serial CLI Terminal	TX0 (GPIO 1), RX0 (GPIO 3)	115200 Baud (USB-Serial Monitor)
Wireless Transceiver	Internal RF Antenna	2.4 GHz ESP-NOW Wireless Mesh/Star
Emergency Pushbutton	GPIO 32	Pull-up Input (Active Low)

ROOM_NODE_1 & ROOM_NODE_2 Pin Mapping (ESP32 #2 & #3)

Component / Function	ESP32 GPIO Pin	Description
PIR Motion Sensor	GPIO 14	Digital Input (Interrupt driven)
Light Relay / Lamp LED	GPIO 12	Digital Output (Active High)
LDR Light Sensor	GPIO 34	Analog Input (ADC1_CH6)
Wireless Transceiver	Internal RF Antenna	2.4 GHz ESP-NOW Telemetry Link
Status LED	GPIO 2	Blinks on wireless packet transmission

3. Wireless ESP-NOW Protocol & Packet Framing (‘common/protocol.h‘)

ESP-NOW transmits variable-length payloads up to 250 bytes encapsulated within 802.11 Vendor-Specific Action Frames. All application packets follow this standardized structure:

```
+---------------+--------+-----------+-----------+----------+-------------+---------+----------
| MAGIC (2B)    | MSG_ID | SENDER_ID | TARGET_ID | MSG_TYPE | PAYLOAD_LEN | PAYLOAD | CHECKSUM
| 0xAA 0x55     | 1 byte |  1 byte   |  1 byte   |  1 byte  |   1 byte    | N bytes |  1 byte
+---------------+--------+-----------+-----------+----------+-------------+---------+----------
```

Peer Device Addressing

- ENTERANCE_MASTER: Target ID 0x01 | Default MAC: 24:0A:C4:00:01:00
- ROOM_NODE_1: Target ID 0x02 | Default MAC: 24:0A:C4:00:02:00
- ROOM_NODE_2: Target ID 0x03 | Default MAC: 24:0A:C4:00:03:00
- Broadcast Address: FF:FF:FF:FF:FF:FF (Used for emergency evacuation alerts)

Message Identifiers:

- 0x01 : MSG_TYPE_HEARTBEAT (Periodic keep-alive ping sent every 2 seconds)
- 0x02 : MSG_TYPE_ROOM_STATUS (Telemetry: uint8_t occupied, uint8_t lights_on, uint16_t lux, uint32_t active_sec)
- 0x03 : MSG_TYPE_LIGHT_OVERRIDE (Master command: 0=Auto, 1=Force OFF, 2=Force ON)

- 0x04 : MSG_TYPE_EMERGENCY_ALERT (Broadcast command: 1=Evacuate, 0=Normal)

Delivery Verification & Reliability

- 1. Hardware MAC Acknowledgement: Every unicast ESP-NOW frame triggers a hardware ACK within the 802.11 MAC layer. The sender's esp_now_register_send_cb callback confirms delivery status (ESP_NOW_SEND_SUCCESS vs ESP_NOW_SEND_FAIL).
- 2. Automatic Retries: If ESP_NOW_SEND_FAIL is detected, the sender retries up to 3 times before raising a communication fault.
- 3. Offline Detection: If ENTERANCE_MASTER receives no heartbeat or telemetry from a room node for more than 10 seconds, it flags that room as OFFLINE on the serial console.

4. Storage Schemas & File Formats (SD Card)

4.1 '/sdcard/users.csv'

Contains the authorized registry of users:

```
tag_id,name,role,is_active
TAG_1001,Kayeera Nathan,admin,1
TAG_1002,Kasozi David,staff,1
TAG_1003,Zziwa Mark,student,1
TAG_1004,Sseku Jovan,student,1
TAG_1005,Namuzibwa Laurinda,staff,1
```

4.2 '/sdcard/access_log.csv'

Appended chronologically on every swipe event:

```
timestamp,tag_id,user_name,direction,decision,reason
2026-10-24 08:15:20,TAG_1001,Kayeera Nathan,ENTRY,GRANTED,Authorized
2026-10-24 21:05:11,TAG_1003,Zziwa Mark,ENTRY,DENIED,Outside Schedule
2026-10-24 21:10:02,TAG_1004,Sseku Jovan,EXIT,DENIED,Curfew Lockdown
2026-10-24 21:15:00,SYSTEM,Admin,EMERGENCY,ACTIVE,Evacuation Triggered
```

4.3 '/sdcard/occupancy.csv' (For Crash Recovery)

Updated dynamically so building occupancy survives unexpected reboots:

```
tag_id,user_name,entry_timestamp
TAG_1001,Kayeera Nathan,2026-10-24 08:15:20
```

4.4 '/sdcard/config.txt'

```
schedule_start=07:00
schedule_end=20:00
lockout_attempts=3
lockout_duration_sec=60
no_motion_timeout_sec=30
heartbeat_interval_sec=2
```

5. Serial CLI Command Specification

Command	Arguments	Permissions	Output / Action
help	none	Public	Displays list of available commands
status	none	Public	Shows system health, wireless room links, SD status
occupancy	none	Public	Lists all persons currently recorded inside
rooms	none	Public	Reports live wireless telemetry from ROOM_NODE_1 & 2
lights	none	Public	Detailed illumination, lux levels & relay states
login	<user> <pass>	Public	Authenticates admin session
logout	none	Authenticated	Terminates active session
schedule show	none	Public	Shows current entry/exit permitted hours
schedule set	<HH:MM> <HH:MM>	Admin Only	Updates building operating hours in RAM & SD
logs	[count]	Admin Only	Dumps the last N log entries
adduser	<tag> <name> <role>	Admin Only	Appends new card to users.csv
deluser	<tag>	Admin Only	Deactivates card in users.csv
energy	none	Admin Only	Displays computed energy savings (kWh)
emergency	‘<on\	off>‘	Admin Only

6. Access Control & Schedule Rules Engine

The decision logic executed on ENTERANCE_MASTER evaluates every tag swipe in the following order:

```
[Tag Scanned]
      |
      v
Is terminal currently locked out (intrusion)?  --YES--> [DENY & Sound Alarm]
      | NO
Is Tag in users.csv & active? ---------------- NO --> [DENY & Increment Invalid Counter]
      | YES (Reset Invalid Counter)
Is current time within Building Schedule?
      |-- NO --> Is User Role == ’admin’?
      |               |-- NO --> [DENY: Outside Schedule Hours (Applies to ENTRY & EXIT)]
      |               |-- YES -> Bypass Schedule Check
      |-- YES -> Proceed to Anti-Passback Check
                      |
Is direction == ENTRY and user already INSIDE? --YES--> [DENY: Anti-Passback Violation]
Is direction == EXIT and user already OUTSIDE? -YES--> [DENY: Anti-Passback Violation]
                      | NO
             [GRANT ACCESS]
       - Trigger appropriate door servo (unlock for 5s)
       - Blink Green LED / Beep Buzzer
       - Update Occupancy Table (Mutex protected)
       - Push event to SD Log Queue
```

PLATFORMIO & VS CODE PROJECT STRUCTURE GUIDE

PLATFORMIO & VS CODE PROJECT STRUCTURE GUIDE

Project: Smart Control and Monitoring System for CoCIS (Makerere University)

Author: Kayeera Nathan (System Architect & Integrator, Group SE-24)

Framework: ESP-IDF (framework = espidf)

Target Platform: Espressif ESP32 (board = esp32dev)

Simulation: Wokwi VS Code Extension + Wokwi Web Simulator

Communication: Wireless ESP-NOW (2.4 GHz Peer-to-Peer)

1. Architectural Directory Layout (Monorepo with Multi-Environment)

This structure is designed specifically for PlatformIO in VS Code. It allows the entire 5-person team to collaborate seamlessly using Git without merge conflicts. The three firmware nodes are designated as ENTERANCE_MASTER, ROOM_NODE_1, and ROOM_NODE_2.

```
SE24_CoCIS_Smart_System/

    .vscode/                                # VS Code Workspace Configuration
        settings.json                       # Editor settings & PlatformIO path
        launch.json                         # Debugging configurations
        extensions.json                     # Recommended extensions (PlatformIO, Wokwi)

    .code-workspace                         # Multi-root VS Code launcher file
    .gitignore                              # Exclude .pio, build artifacts, temp files
    platformio.ini                          # Central multi-environment PlatformIO config
    wokwi.toml                              # Wokwi VS Code extension configuration
    diagram.json                            # 3-ESP32 Wokwi circuit simulation schematic
    CMakeLists.txt                          # Root ESP-IDF CMakeLists

    components/                             # Shared ESP-IDF Components & Libraries
        common_protocol/                   # Shared Packet Definitions & Concurrency Headers
            CMakeLists.txt                  # ESP-IDF component registration
            include/
                protocol.h                  # ESP-NOW packet frames & message types
                sync_objects.h              # FreeRTOS mutexes & queue handles
                config_defaults.h          # Default timings, pin maps & thresholds
                mac_address_map.h           # Virtual MAC addresses of all 3 nodes
            src/
                protocol.c                  # Checksum calculation & packet serialization
                sync_objects.c              # Mutex & queue initializer helpers

    src/                                    # Firmware Source Code (Split by Node)

        ENTERANCE_MASTER/                   # ENTERANCE_MASTER: Entrance Terminal & Gateway (E
                                            # Owners: Nathan (Lead), David (Entrance), Jovan (
            main.c                          # Master app_main(), task spawning, system state
            rfid_manager.c                  # SPI RC522 reader handling (Entry & Exit)
            rfid_manager.h
            door_actuator.c                 # Servo PWM control & 5s timed relock
            door_actuator.h
            schedule_enforcer.c             # 07:00-20:00 schedule check & curfew lockdown
            schedule_enforcer.h
            anti_passback.c                 # State-based anti-passback validation
            anti_passback.h
            sd_storage.c                    # FATFS file operations (users, logs, config)
            sd_storage.h
            serial_cli.c                    # Administrative terminal parser & menus
            serial_cli.h
```

```
            auth_security.c                 # Password hashing & session timeout
            auth_security.h
            esp_now_gateway.c               # ESP-NOW receiver callback & heartbeat monitor
            esp_now_gateway.h

        ROOM_NODE_1/                        # ROOM_NODE_1: Lecture Room 1 Terminal (ESP32 #2)
                                            # Owner: Zziwa Mark Alex
            main.c                          # Room 1 app_main() & event loop
            motion_sensor.c                 # PIR interrupt, debounce & 30s countdown
            motion_sensor.h
            light_sensor.c                  # ADC LDR reading & daylight-aware thresholding
            light_sensor.h
            light_relay.c                   # GPIO relay control & manual override
            light_relay.h
            esp_now_sender.c                # 2-second telemetry telemetry dispatch
            esp_now_sender.h

        ROOM_NODE_2/                        # ROOM_NODE_2: Lecture Room 2 Terminal (ESP32 #3)
                                            # Owner: Zziwa Mark Alex
            main.c                          # Room 2 app_main() & event loop
            motion_sensor.c                 # Modular PIR driver
            motion_sensor.h
            light_sensor.c                  # Modular LDR driver
            light_sensor.h
            light_relay.c                   # Modular Relay driver
            light_relay.h
            esp_now_sender.c                # Telemetry sender with Room 2 MAC/ID
            esp_now_sender.h

    simulation/                             # Simulation Assets & Virtual Storage
        sdcard/                             # Pre-populated virtual MicroSD files
            users.csv                       # Registered tags (Nathan, David, Mark, Jovan, Lau
            access_log.csv                  # Historical audit log sample
            occupancy.csv                   # Crash recovery state file
            config.txt                      # Initial system configuration

    docs/                                   # Documentation, Schematics & Report Drafts
        architecture/                       # SYSTEM_ARCHITECTURE_DIAGRAM.svg & .png
        interface_contract/                 # INTERFACE_CONTRACT_AND_DESIGN_WIRELESS.pdf
        test_matrix/                        # Test cases & bug tracking spreadsheet
        report/                             # LaTeX / Word report source & member contribution
```

2. Core Configuration Files (Copy-Paste Ready)

2.1 ‘platformio.ini‘ (Central Multi-Target Configuration)

Place this file in the root directory. It configures PlatformIO to build three distinct firmware binaries: ENTERANCE_MASTER, ROOM_NODE_1, and ROOM_NODE_2:

```
; ========================================================================
; PlatformIO Project Configuration for SE-24 CoCIS Smart System
; Multi-Target ESP-IDF Setup: ENTERANCE_MASTER, ROOM_NODE_1, ROOM_NODE_2
; ========================================================================

[platformio]
default_envs = ENTERANCE_MASTER

[env]
platform = espressif32 @ 6.5.0
framework = espidf
```

```
board = esp32dev
monitor_speed = 115200
monitor_filters = direct, esp32_exception_decoder

; ------------------------------------------------------------------------
; Target 1: ENTERANCE_MASTER - Entrance Master & Gateway (ESP32 #1)
; ------------------------------------------------------------------------
[env:ENTERANCE_MASTER]
build_src_filter = +<ENTERANCE_MASTER/*>
build_flags =
    -D NODE_ENTERANCE_MASTER=1
    -D CORE_DEBUG_LEVEL=3
    -I components/common_protocol/include

; ------------------------------------------------------------------------
; Target 2: ROOM_NODE_1 - Lecture Room 1 Terminal (ESP32 #2)
; ------------------------------------------------------------------------
[env:ROOM_NODE_1]
build_src_filter = +<ROOM_NODE_1/*>
build_flags =
    -D NODE_ROOM_1=1
    -D ROOM_ID=1
    -D CORE_DEBUG_LEVEL=3
    -I components/common_protocol/include

; ------------------------------------------------------------------------
; Target 3: ROOM_NODE_2 - Lecture Room 2 Terminal (ESP32 #3)
; ------------------------------------------------------------------------
[env:ROOM_NODE_2]
build_src_filter = +<ROOM_NODE_2/*>
build_flags =
    -D NODE_ROOM_2=1
    -D ROOM_ID=2
    -D CORE_DEBUG_LEVEL=3
    -I components/common_protocol/include
```

2.2 ‘wokwi.toml‘ (Multi-Node Wokwi Simulation Config)

Place this in the root directory. It connects the VS Code Wokwi extension to your build outputs:

```
[wokwi]
version = 1
diagram = "diagram.json"
firmware = ".pio/build/ENTERANCE_MASTER/firmware.bin"
elf = ".pio/build/ENTERANCE_MASTER/firmware.elf"

# Multi-node simulation chips:
[[wokwi.chips]]
id = "enterance_master_esp"
firmware = ".pio/build/ENTERANCE_MASTER/firmware.bin"
elf = ".pio/build/ENTERANCE_MASTER/firmware.elf"

[[wokwi.chips]]
id = "room_node_1_esp"
firmware = ".pio/build/ROOM_NODE_1/firmware.bin"
elf = ".pio/build/ROOM_NODE_1/firmware.elf"

[[wokwi.chips]]
id = "room_node_2_esp"
firmware = ".pio/build/ROOM_NODE_2/firmware.bin"
elf = ".pio/build/ROOM_NODE_2/firmware.elf"
```

2.3 ‘diagram.json‘ (3-Board Cable-Free Wokwi Circuit)

Because we use ESP-NOW, there are no physical serial wires between the boards:

```
{
  "version": 1,
  "author": "Kayeera Nathan (Group SE-24)",
  "editor": "wokwi",
  "parts": [
    { "type": "board-esp32-devkit-c-v4", "id": "enterance_master_esp", "top": 0, "left": 0, "at
    { "type": "board-esp32-devkit-c-v4", "id": "room_node_1_esp",      "top": 0, "left": 420, "
    { "type": "board-esp32-devkit-c-v4", "id": "room_node_2_esp",      "top": 0, "left": 840, "

    { "type": "wokwi-servo", "id": "servo_entry", "top": 180, "left": -120, "attrs": {} },
    { "type": "wokwi-servo", "id": "servo_exit",  "top": 260, "left": -120, "attrs": {} },
    { "type": "wokwi-led",   "id": "led_green",   "top": 350, "left": -80,  "attrs": { "color":
    { "type": "wokwi-led",   "id": "led_red",     "top": 350, "left": -40,  "attrs": { "color":
    { "type": "wokwi-buzzer","id": "buzzer1",     "top": 400, "left": -80,  "attrs": {} },

    { "type": "wokwi-pir-motion-sensor", "id": "pir1", "top": 180, "left": 400, "attrs": {} },
    { "type": "wokwi-photoresistor-sensor", "id": "ldr1", "top": 260, "left": 400, "attrs": {}
    { "type": "wokwi-relay-module", "id": "relay1", "top": 340, "left": 400, "attrs": {} },

    { "type": "wokwi-pir-motion-sensor", "id": "pir2", "top": 180, "left": 820, "attrs": {} },
    { "type": "wokwi-photoresistor-sensor", "id": "ldr2", "top": 260, "left": 820, "attrs": {}
    { "type": "wokwi-relay-module", "id": "relay2", "top": 340, "left": 820, "attrs": {} }
  ],
  "connections": [
    [ "enterance_master_esp:13", "servo_entry:PWM", "orange", [ "v0" ] ],
    [ "enterance_master_esp:15", "servo_exit:PWM",  "orange", [ "v0" ] ],
    [ "enterance_master_esp:2",  "led_green:A",     "green",  [ "v0" ] ],
    [ "enterance_master_esp:4",  "led_red:A",       "red",    [ "v0" ] ],
    [ "enterance_master_esp:12", "buzzer1:1",       "purple", [ "v0" ] ],

    [ "room_node_1_esp:14",      "pir1:OUT",        "blue",   [ "v0" ] ],
    [ "room_node_1_esp:34",      "ldr1:AO",         "gold",   [ "v0" ] ],
    [ "room_node_1_esp:12",      "relay1:IN",       "green",  [ "v0" ] ],

    [ "room_node_2_esp:14",      "pir2:OUT",        "blue",   [ "v0" ] ],
    [ "room_node_2_esp:34",      "ldr2:AO",         "gold",   [ "v0" ] ],
    [ "room_node_2_esp:12",      "relay2:IN",       "green",  [ "v0" ] ]
  ]
}
```

2.4 ‘CMakeLists.txt‘ for ESP-IDF in PlatformIO

Root ‘CMakeLists.txt‘:

```
cmake_minimum_required(VERSION 3.16)
include($ENV{IDF_PATH}/tools/cmake/project.cmake)
project(cocis_smart_system)
```

‘components/common_protocol/CMakeLists.txt‘:

```
idf_component_register(
    SRCS "src/protocol.c" "src/sync_objects.c"
    INCLUDE_DIRS "include"
    REQUIRES nvs_flash esp_wifi esp_netif esp_now freertos
)
```

2.5 '.gitignore' (Keep the Repo Clean)

```
.pio/
.vscode/.browse.c_cpp.db*
.vscode/c_cpp_properties.json
build/
*.bin
*.elf
*.map
__pycache__/
*.pyc
.DS_Store
Thumbs.db
```

3. Team Member Directory & Git Ownership Matrix

Member	Assigned Directory / Files	Git Branch	Core Responsibilities
Kayeera Nathan	components/common_protocol/, src/ENTERANCE_MASTER/main.c, esp_now_gateway.*, platformio.ini, wokwi.toml	main, feat/arch-nathan	System architecture, FreeRTOS queues/mutexes, ESP-NOW protocol, emergency mode, integrator.
Kasozi David Paul	src/ENTERANCE_MASTER/rfid_manager.*, door_actuator.*, anti_passback.*	feat/entrance-david	RFID read drivers, servo door opening, anti-passback rule, intrusion lockout.
Zziwa Mark Alex	src/ROOM_NODE_1/, src/ROOM_NODE_2/	feat/rooms-mark	PIR motion sensing, 30s timer, LDR daylight-aware lighting, relay actuators, telemetry sender.
Sseku Jovan	src/ENTERANCE_MASTER/sd_storage.*, simulation/sdcard/	feat/storage-jovan	SPI MicroSD driver, FATFS mounting, CSV logs, crash recovery from reboot.
Namuzibwa Laurinda	src/ENTERANCE_MASTER/serial_cli.*, auth_security.*, docs/test_matrix/	feat/cli-laurinda	Command parser loop, SHA-256 login, schedule set command, test plan & QA.

4. How to Open & Build in VS Code (Step-by-Step)

- 1. Install Prerequisites in VS Code:
- Install PlatformIO IDE extension (platformio.platformio-ide).
- Install Wokwi Simulator extension (wokwi.wokwi-vscode).
- 2. Open Project Folder:
- Launch VS Code $\rightarrow$ File $\rightarrow$ Open Folder... $\rightarrow$ Select SE24_CoCIS_Smart_System.
- 3. Compile a Specific Target:
- Click the PlatformIO Alien icon on the left sidebar.
- Under Project Tasks, select:
- ENTERANCE_MASTER $\rightarrow$ Click Build
- ROOM_NODE_1 $\rightarrow$ Click Build
- ROOM_NODE_2 $\rightarrow$ Click Build
- Or run in terminal: pio run -e ENTERANCE_MASTER
- 4. Run Live Wokwi Simulation:
- Press F1 (or Ctrl+Shift+P) in VS Code.
- Type Wokwi: Start Simulator and press Enter.

- Wokwi will load diagram.json and start all three ESP32 microcontrollers in browser simulation with live wireless communication!

COURSEWORK DELIVERABLES & DEFENSE PREPARATION GUIDE

COURSEWORK DELIVERABLES & DEFENSE PREPARATION GUIDE

Student: Kayeera Nathan (Group Leader & System Architect, SE-24)

Project: Smart Control and Monitoring System for CoCIS, Makerere University

Communication Bus: Wireless ESP-NOW (2.4 GHz Peer-to-Peer Wi-Fi)

Target Deadline: 2 November 2026

1. Overview of Required Deliverables (Per Coursework Brief)

According to Section 4 of the official coursework specification, your group must submit a comprehensive package consisting of:

- 1. System Operation Report (PDF on MUELE):
- Architecture, hardware pinout, software architecture, FreeRTOS synchronization, wireless ESP-NOW protocol, and feature documentation.
- Individual contributions breakdown.
- Group member details.
- 2. Wokwi Simulation Link: Live, publicly accessible simulation demonstrating the multi-board wireless system.
- 3. Individual Video Demonstrations: Hyperlinks to 5 individual screen recordings (3-5 minutes each).
- 4. Viva / Oral Presentation: Monday, 2 November 2026 before the lecturer (attendance compulsory for marks).

2. Deliverable 1: Report Structure & Author Assignments

To ensure maximum marks under the assessment criteria, structure the final report as follows:

```
Title Page: Project Title, Course, Academic Year, Date, Group SE-24 Details Table
1. Executive Summary & Problem Background ( Nathan )
2. System Architecture & Distributed Wireless Topology ( Nathan )
   - 3-ESP32 Distributed Architecture
   - Peer-to-Peer 2.4 GHz ESP-NOW Wireless Mesh/Star
   - Hardware Pin Mapping & Circuit Schematics
3. Communication & Synchronization Protocols ( Nathan )
   - ESP-NOW Connectionless Packet Framing & Delivery ACKs
   - FreeRTOS Mutexes, Semaphores & Message Queues
   - Race Condition & Concurrency Prevention
4. Entrance Terminal & Access Control Subsystem ( David )
   - RFID Reading & Door Mechanism (Servos)
   - Schedule Enforcement & Anti-Passback Algorithm
5. Lecture Room Automated Lighting Subsystem ( Mark )
   - PIR Motion Sensing, Debounce & Timers
   - Ambient Light Thresholding (LDR) & ESP-NOW Telemetry Dispatch
6. Data Storage & System Reliability Subsystem ( Jovan )
   - SPI SD Card Implementation & FATFS Filesystem
   - CSV Schemas & Crash Recovery from Power Failure
7. Serial Command Interface & Security Management ( Laurinda )
   - Command Parser & Navigation Menus
   - Authentication (Hashed Passwords, Session Timeout, Lockout)
```

```
8. Innovative Features & System Value-Add ( Nathan & All )
   - Industrial Wireless IoT Network (ESP-NOW) over Cable-based Serial (Nathan)
   - Wireless Emergency Evacuation Broadcast (Nathan)
   - Wireless Heartbeat & Node Failure Detection (Nathan)
   - Anti-Passback Security (David)
   - Daylight-Aware Energy Saving Logic (Mark)
   - Crash Recovery & Non-Volatile State Restoration (Jovan)
   - Serial Intrusion Lockout (Laurinda)
9. Verification, Test Results & Concurrency Evaluation ( Laurinda & All )
   - Comprehensive Test Matrix (Inputs, Expected vs. Observed Outputs)
10. System Links, Contributions & Group Member Declarations
   - Public Wokwi Project Link
   - Individual Video Links Table
   - Itemized Individual Contributions Table
   - Declaration of Originality
```

3. Deliverable 2: Wokwi Multi-ESP32 Wireless Simulation Setup

In Wokwi, you can simulate multiple microcontrollers communicating via ESP-NOW in a single project. Because ESP-NOW operates over the simulated 2.4 GHz radio space, no physical RX/TX interconnecting wires are required between the boards!

Clean Wokwi ‘diagram.json‘ Setup:

```
{
  "version": 1,
  "author": "Kayeera Nathan",
  "editor": "wokwi",
  "parts": [
    { "type": "board-esp32-devkit-c-v4", "id": "enterance_master_esp", "top": 0, "left": 0, "at
    { "type": "board-esp32-devkit-c-v4", "id": "room_node_1_esp",      "top": 0, "left": 420, "
    { "type": "board-esp32-devkit-c-v4", "id": "room_node_2_esp",      "top": 0, "left": 840, "
  ],
  "connections": []
}
```

TIP / BEST PRACTICE
Why Wokwi Multi-ESP32 Wireless is Elegant: Each ESP32 board initializes Wi-Fi in Station mode and uses ‘esp_now_send()‘ with the target’s MAC address. Wokwi’s virtual RF environment routes packets directly between the simulated ESP32 instances.

4. Deliverable 3: Nathan’s Individual Video Demonstration Script

Every student must submit a 3 to 5 minute video. Marks are awarded for clarity and demonstration of code you wrote.

Recommended Video Outline (4 Minutes Total):

- 0:00 - 0:45 (Introduction & Architecture Overview):
- "Hello, I am Kayeera Nathan, registration number 24/U/25913/EVE. I am the group leader, system architect, and integrator for group SE-24. Our project is the Smart Control and Monitoring System for CoCIS."
- "Our architecture is distributed across three ESP32 nodes communicating wirelessly over ESP-NOW, Espressif's connectionless 2.4 GHz Wi-Fi protocol, eliminating the need for hundreds of meters of fragile serial cabling across university corridors."
- 0:45 - 2:00 (Architecture, Synchronization & FreeRTOS Walkthrough):
- Display common/protocol.h and sync_manager.c.

- Explain: "Under Section 2.7, resource synchronization is paramount. I designed the FreeRTOS concurrency model where incoming wireless packets are pushed into xWirelessPacketQueue directly from the ESP-NOW receive callback ISR, preventing task starvation. Shared state-including our building occupancy table and operating schedule-is guarded by mutexes (xOccupancyMutex, xScheduleMutex) to eliminate race conditions."
- 2:00 - 3:00 (Wireless ESP-NOW Protocol & Node Heartbeat):
- Show the C code in esp_now_transport.c.
- Explain the 2.4 GHz packet framing, MAC addressing, and hardware delivery status callbacks.
- Demonstrate the master's periodic heartbeat task that monitors Room 1 and Room 2 telemetry every 2 seconds and flags an alert if a wireless node drops offline.
- 3:00 - 4:15 (Live Wokwi Demonstration of Wireless Emergency Evacuation):
- Switch to the live Wokwi browser tab.
- Run the simulation; type status or rooms in the master terminal to show live wireless telemetry streaming from the lecture rooms.
- Trigger emergency on in the terminal.
- Point out: "Observe that both entrance door servos instantly swing to 90 degrees open. Simultaneously, an emergency broadcast packet is transmitted over ESP-NOW, causing both lecture room microcontrollers to override their lights ON for occupant evacuation, and an emergency event is logged to the SD card."
- 4:15 - 4:30 (Conclusion):
- Summarize the reliability and innovations of the system.

5. Defense & Presentation Q&A Preparation

The lecturer will question each member individually on Monday, 2 November. As Group Leader and Architect, expect questions like:

- 1. "Why did you choose ESP-NOW over traditional UART serial cables?"
- *Answer:* "In a campus building like CoCIS, routing physical RS232/UART serial cables from the main entrance to multiple lecture rooms across floors is expensive, susceptible to electromagnetic interference, and difficult to scale. ESP-NOW provides a low-latency, cable-free 2.4 GHz wireless solution with built-in hardware MAC-layer delivery confirmation and zero reliance on external network infrastructure."
- 2. "Why did you use ESP-NOW instead of standard Wi-Fi with MQTT or HTTP?"
- *Answer:* "Standard Wi-Fi requires connecting to a router (SSID/password), dynamic IP allocation via DHCP, and an external MQTT broker or cloud server. If campus Wi-Fi or internet goes down, building access and safety would be compromised. ESP-NOW is completely connectionless and operates peer-to-peer at the 802.11 MAC layer, guaranteeing sub-5ms latency and full offline autonomy."
- 3. "How does your system prevent race conditions when a wireless packet arrives during an SD card write?"
- *Answer:* "The ESP-NOW receive callback executes in an interrupt context. Instead of performing heavy operations inside the callback, it copies the packet into a FreeRTOS queue (xWirelessPacketQueue) using xQueueSendFromISR(). A separate worker task consumes the queue and acquires xRoomStateMutex before updating shared memory. Meanwhile, the SD card logger runs in its own task guarded by xSDCardMutex, completely decoupling storage I/O from wireless reception."
- 4. "How does your system enforce that people inside cannot leave outside hours?"
- *Answer:* "The exit door logic checks both the schedule and the user's role. If a student attempts to scan out after curfew (e.g. 20:00), the state machine evaluates current_time > schedule_end. Unless the tag has administrative credentials, the exit servo remains locked and the event is flagged in access_log.csv as an unauthorized after-hours attempt."